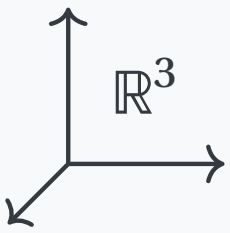
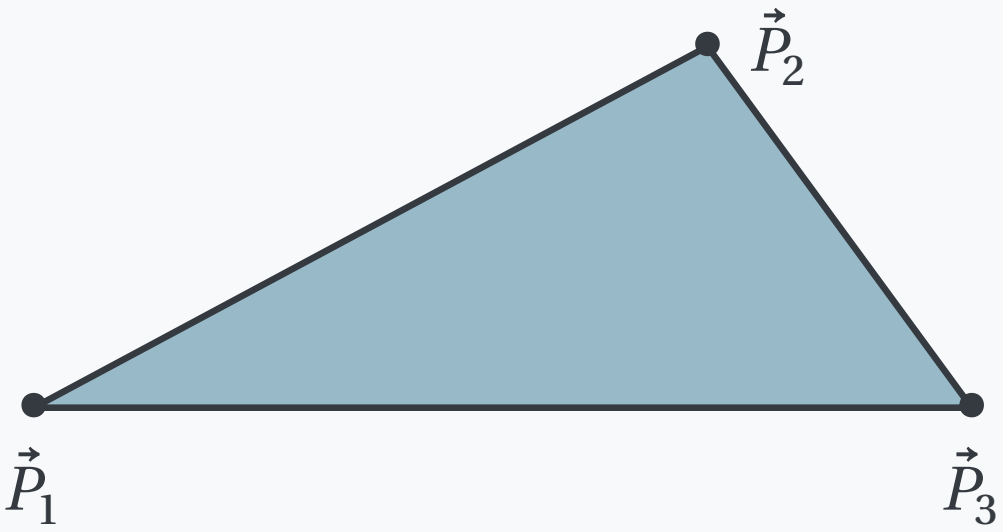
Curved Point-Normal Triangles.

March 31, 2026

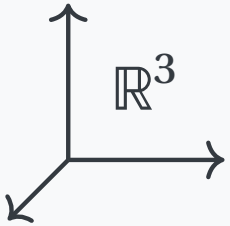
Hugo SALOU

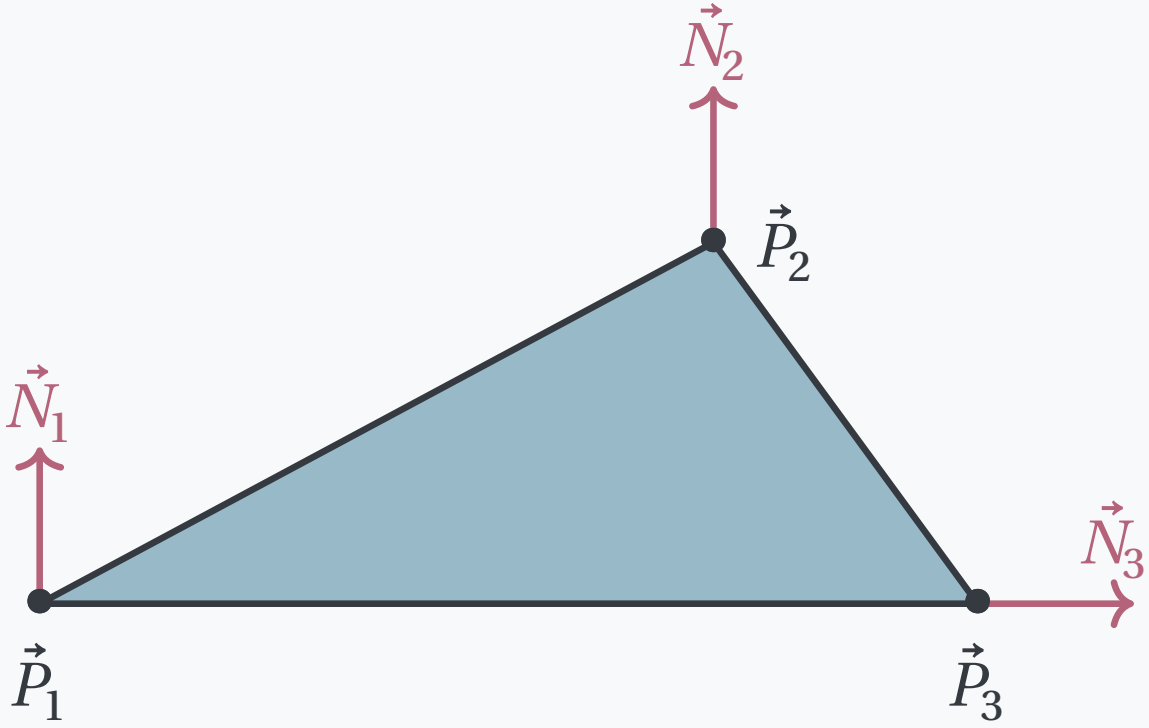
Point-Normal triangle.



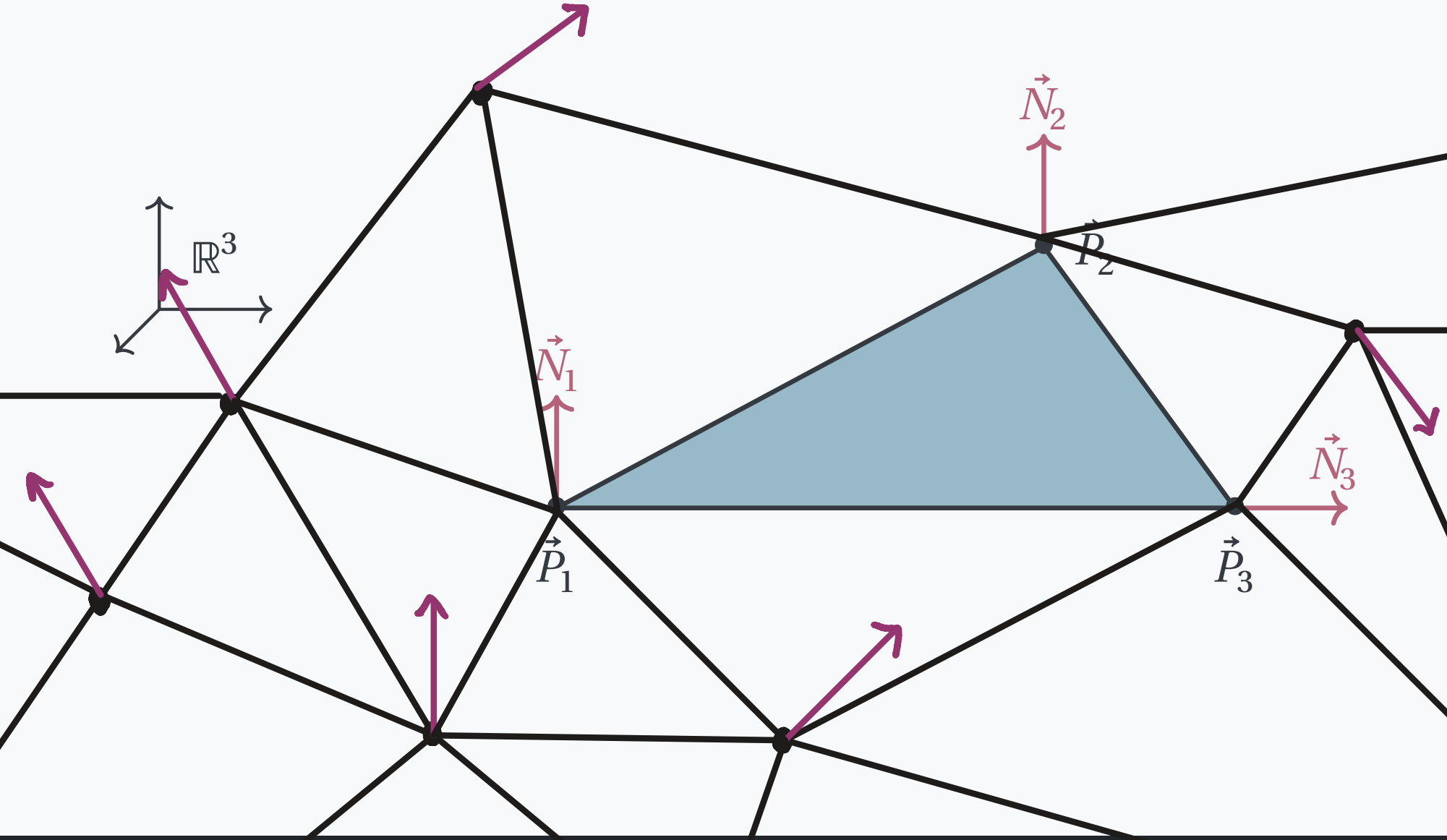


Point-Normal triangle.

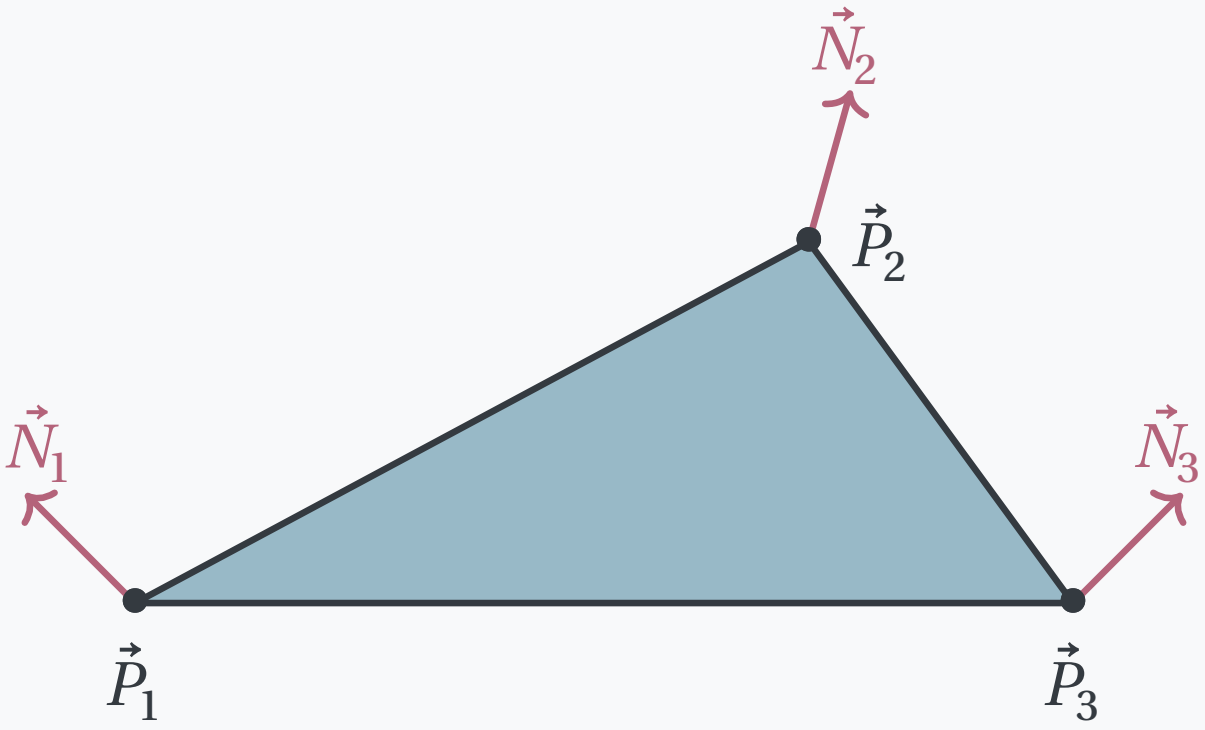




Point-Normal ~~triangle~~ mesh.

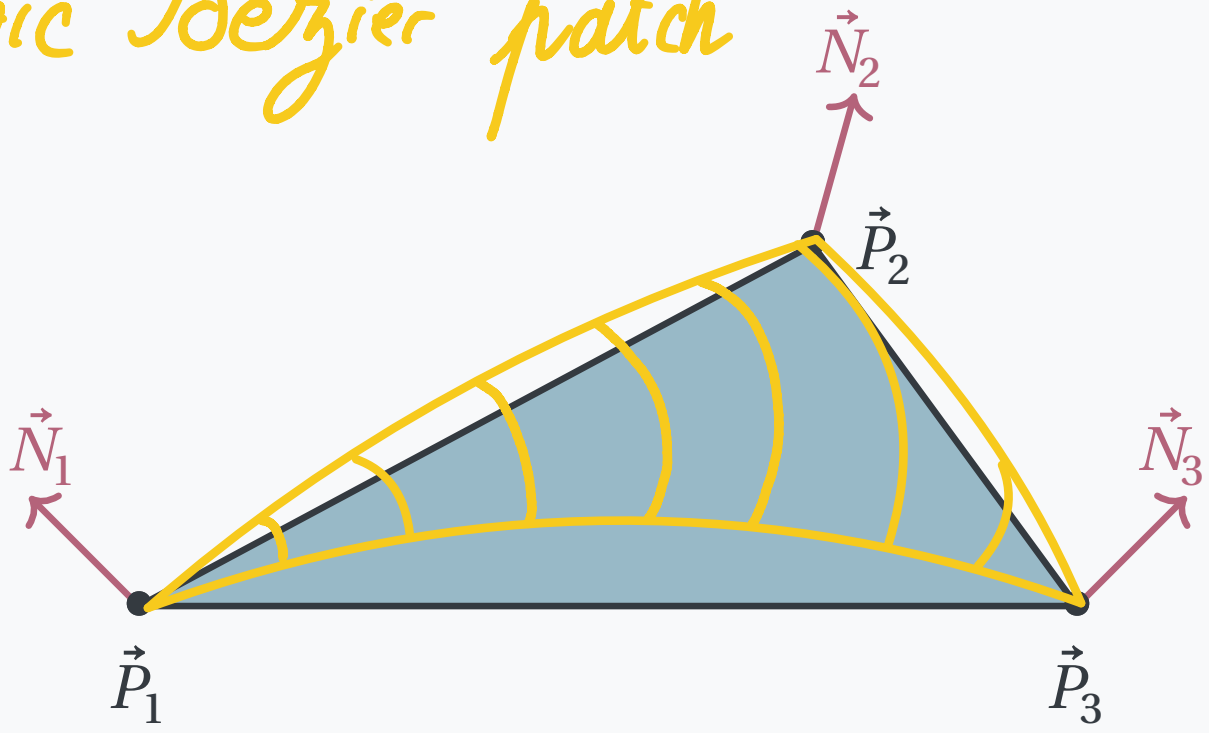


Curved PN triangle.



Curved PN triangle.

Cubic Bézier patch



An explicit formula.

$$\vec{b} : \left| \begin{array}{rcl} \Delta & \longrightarrow & \mathbb{R}^3 \\ (u, v, w) & \longmapsto & \sum_{i+j+k=3} \vec{b}_{kij}\, u^i v^j w^k, \end{array} \right.$$

$$\vec{b}_{300} := \vec{P}_1, \quad \vec{b}_{030} := \vec{P}_2, \quad \vec{b}_{003} := \vec{P}_3, \quad w_{ij} := (\vec{P}_j - \vec{P}_i) \cdot \vec{N}_i$$

$$\vec{b}_{012} := \frac{1}{3}(2\vec{P}_3 + \vec{P}_2 - w_{32}\vec{N}_3), \quad \vec{b}_{021} := \frac{1}{3}(2\vec{P}_2 + \vec{P}_3 - w_{23}\vec{N}_2),$$

$$\vec{b}_{102} := \frac{1}{3}(2\vec{P}_3 + \vec{P}_1 - w_{31}\vec{N}_3), \quad \vec{b}_{201} := \frac{1}{3}(2\vec{P}_1 + \vec{P}_3 - w_{13}\vec{N}_1),$$

$$\vec{b}_{120} := \frac{1}{3}(2\vec{P}_2 + \vec{P}_1 - w_{21}\vec{N}_2), \quad \vec{b}_{210} := \frac{1}{3}(2\vec{P}_1 + \vec{P}_2 - w_{12}\vec{N}_1),$$

$$\vec{b}_{111} := \alpha\vec{E} + (1-\alpha)\vec{V} \qquad \vec{V} := \frac{1}{3}(\vec{P}_1 + \vec{P}_2 + \vec{P}_3)$$

$$\vec{E} := \frac{1}{6}(\vec{b}_{012} + \vec{b}_{021} + \vec{b}_{102} + \vec{b}_{201} + \vec{b}_{120} + \vec{b}_{210}).$$

An explicit formula.

Control points

$$\vec{b} : \left| \begin{array}{rcl} \Delta & \longrightarrow & \mathbb{R}^3 \\ (u, v, w) & \longmapsto & \sum_{i+j+k=3} \vec{b}_{kij}\, u^i v^j w^k, \end{array} \right.$$

$$\vec{b}_{300} := \vec{P}_1, \quad \vec{b}_{030} := \vec{P}_2, \quad \vec{b}_{003} := \vec{P}_3, \quad w_{ij} := (\vec{P}_j - \vec{P}_i) \cdot \vec{N}_i$$

$$\vec{b}_{012} := \frac{1}{3}(2\vec{P}_3 + \vec{P}_2 - w_{32}\vec{N}_3), \quad \vec{b}_{021} := \frac{1}{3}(2\vec{P}_2 + \vec{P}_3 - w_{23}\vec{N}_2),$$

$$\vec{b}_{102} := \frac{1}{3}(2\vec{P}_3 + \vec{P}_1 - w_{31}\vec{N}_3), \quad \vec{b}_{201} := \frac{1}{3}(2\vec{P}_1 + \vec{P}_3 - w_{13}\vec{N}_1),$$

$$\vec{b}_{120} := \frac{1}{3}(2\vec{P}_2 + \vec{P}_1 - w_{21}\vec{N}_2), \quad \vec{b}_{210} := \frac{1}{3}(2\vec{P}_1 + \vec{P}_2 - w_{12}\vec{N}_1),$$

$$\vec{b}_{111} := \alpha\vec{E} + (1 - \alpha)\vec{V} \qquad \vec{V} := \frac{1}{3}(\vec{P}_1 + \vec{P}_2 + \vec{P}_3)$$

$$\vec{E} := \frac{1}{6}(\vec{b}_{012} + \vec{b}_{021} + \vec{b}_{102} + \vec{b}_{201} + \vec{b}_{120} + \vec{b}_{210}).$$

A few examples.

demo

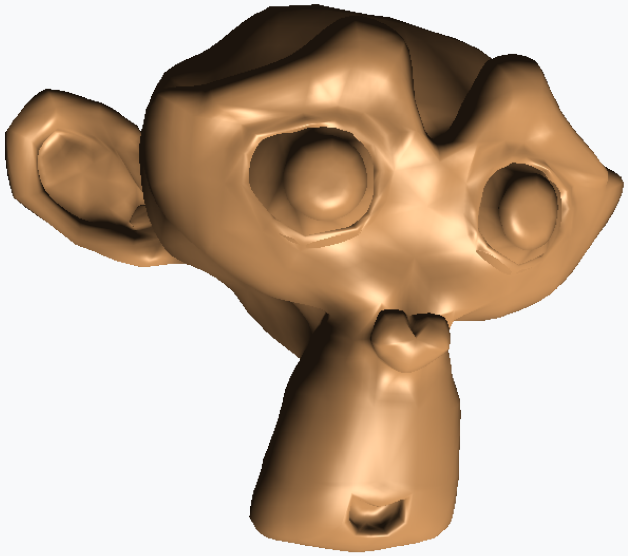
```
python3 object.py data/suzanne-500.ply
```

Computing the normals: linear model.

$$\vec{n}(u, v, w) := \text{normalized}\left(w\vec{N}_1 + u\vec{N}_2 + v\vec{N}_3\right)$$

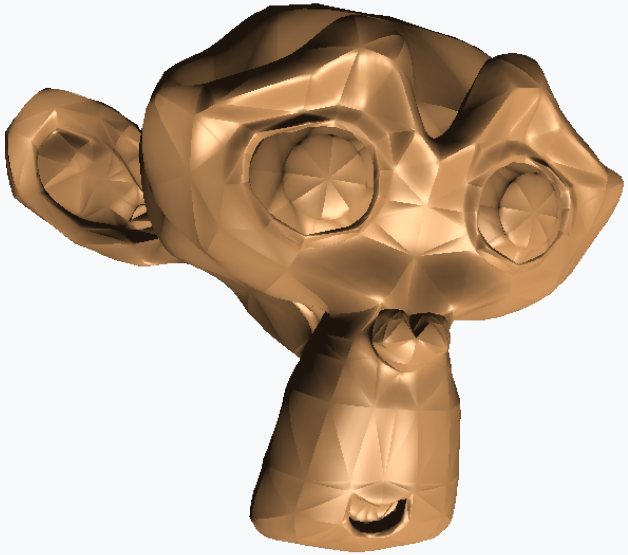
Computing the normals: quadratic model.

$$\vec{n}(u,v,w) := \text{normalized}\Big(\sum_{i+j+k=2} \vec{n}_{kij} u^i v^j w^k\Big)$$



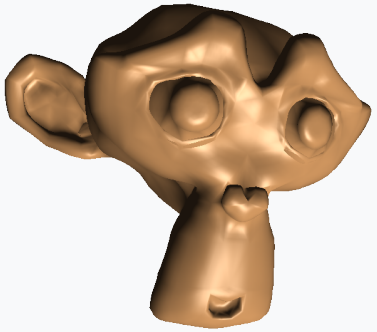
Computing the normals: “exact” model.

$$\vec{n}(u,v,w) := \text{normalized}\left(\frac{\partial \vec{b}(u,v,w)}{\partial u} \times \frac{\partial \vec{b}(u,v,w)}{\partial v}\right)$$

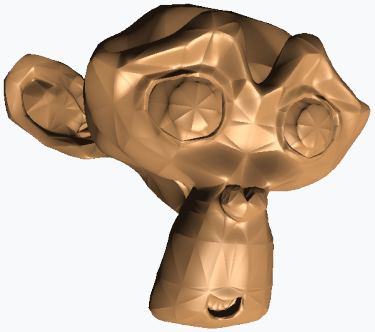


Comparing the models.

(a) Linear varying normals



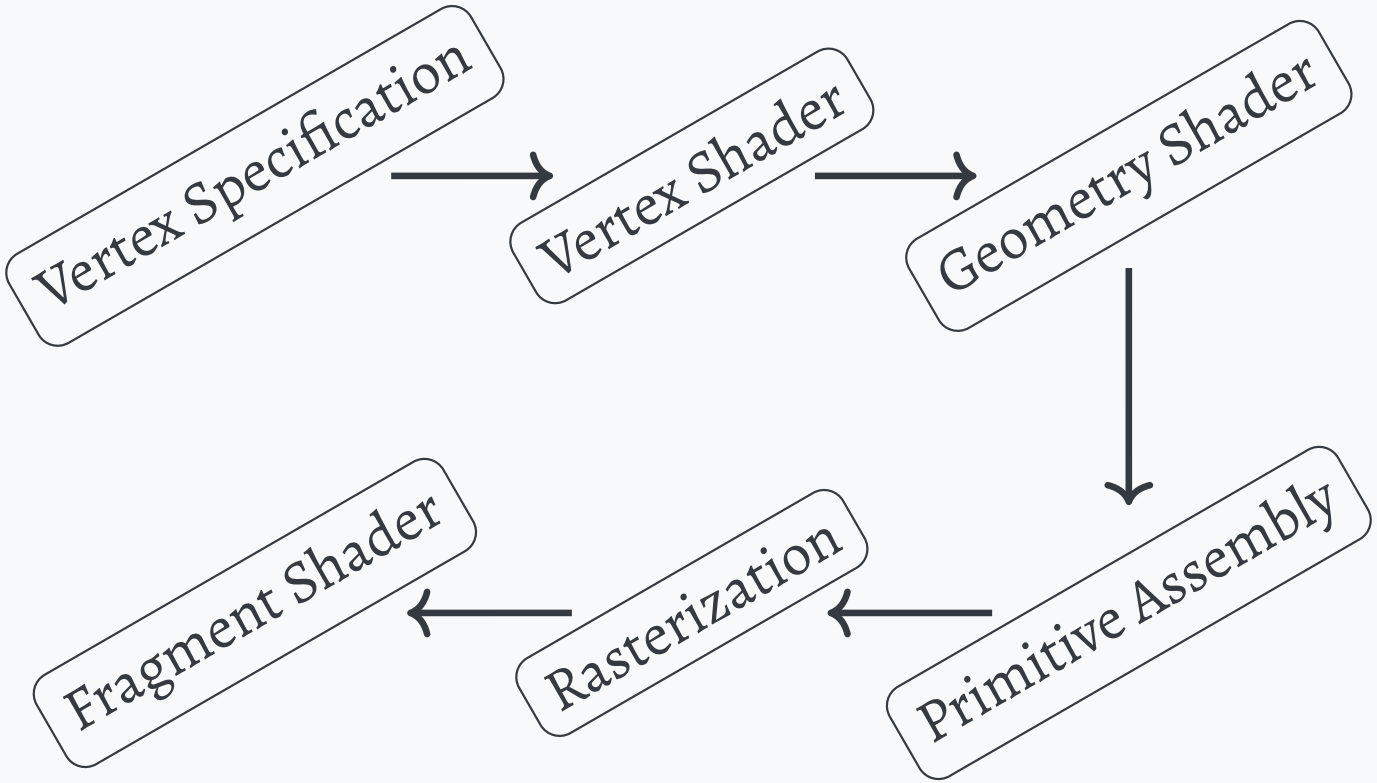
(b) Quadratic varying normals



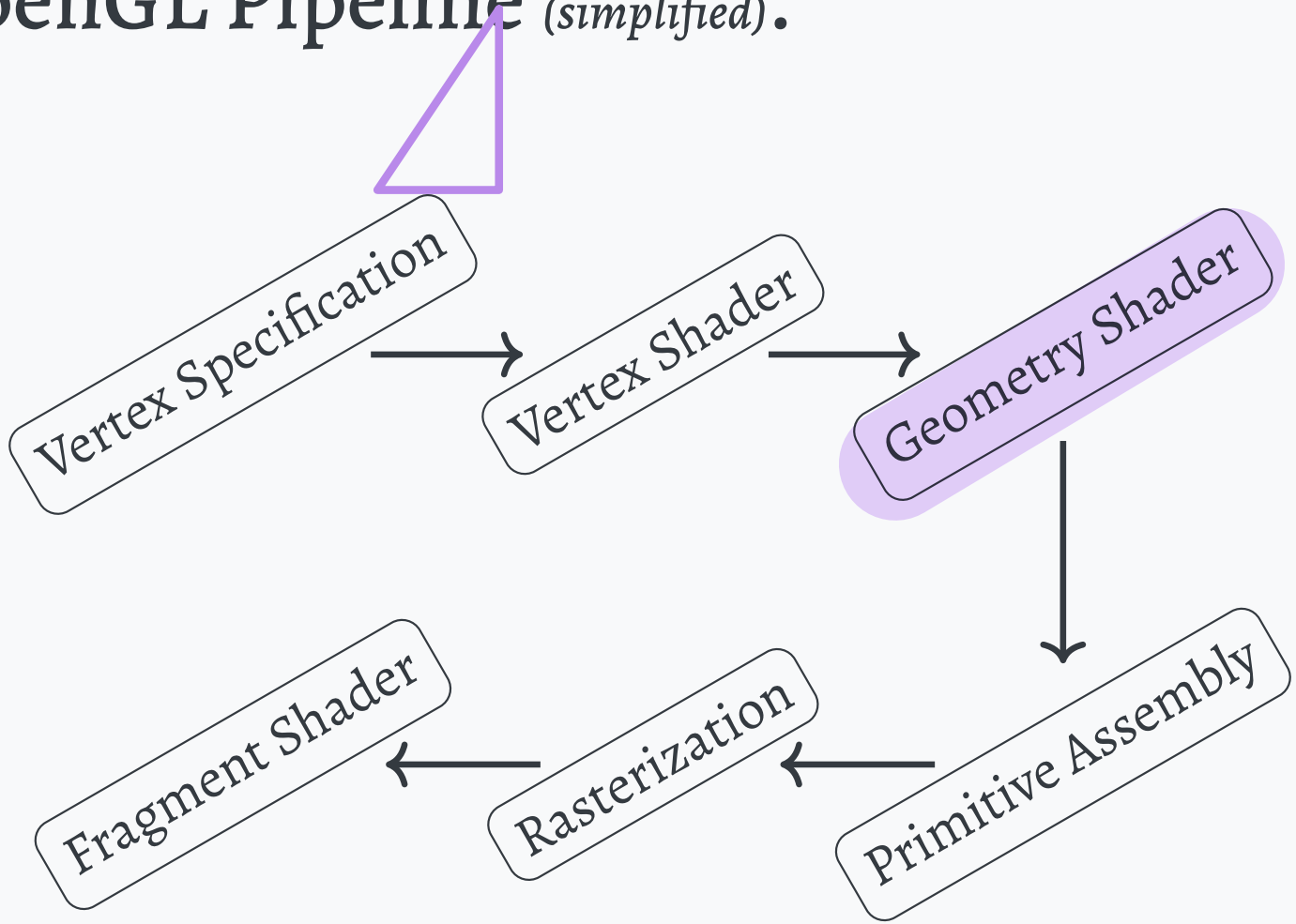
(c) “Exact” normals

Figure: Comparison of different normals-computing methods

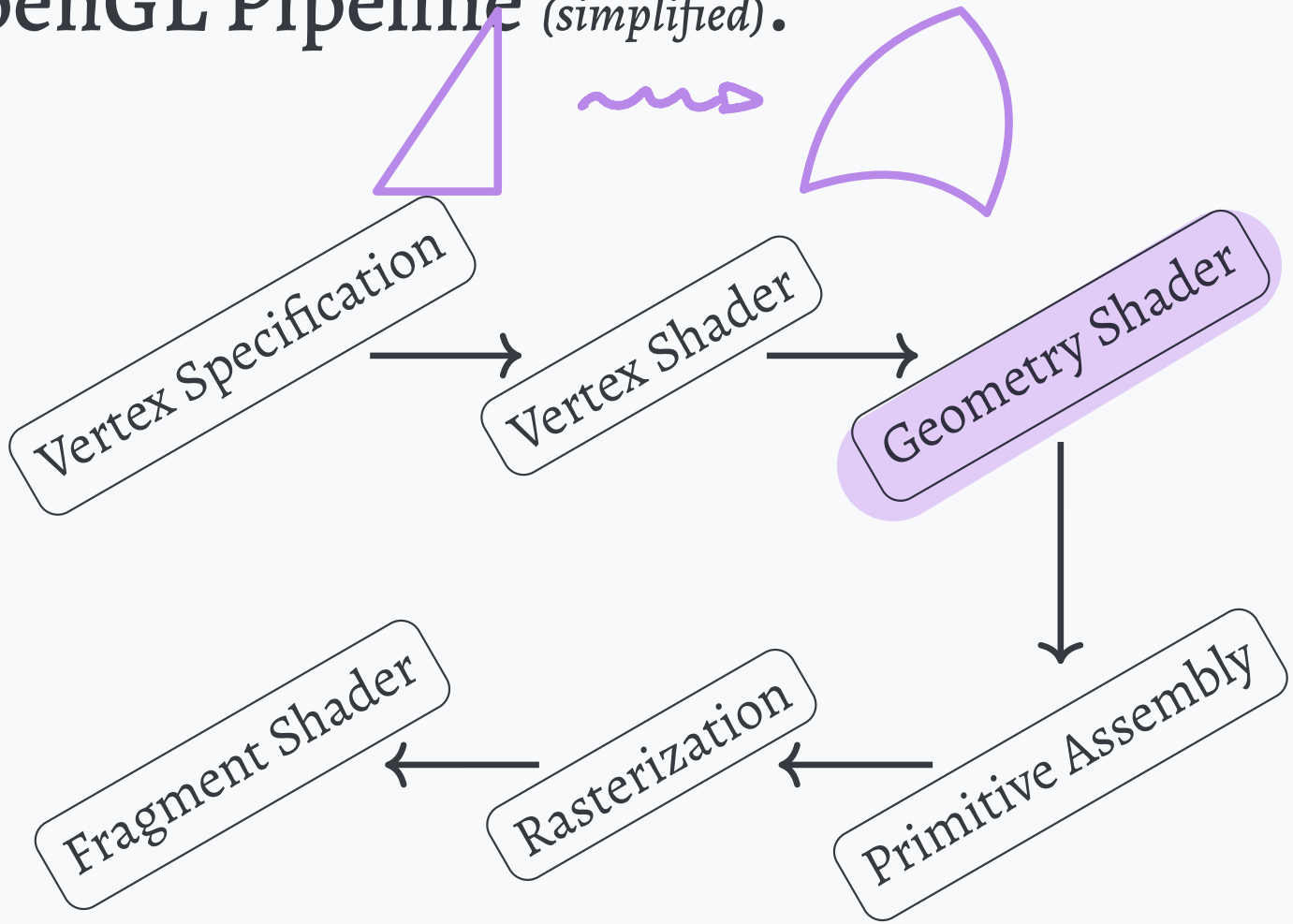
The OpenGL Pipeline *(simplified)*.



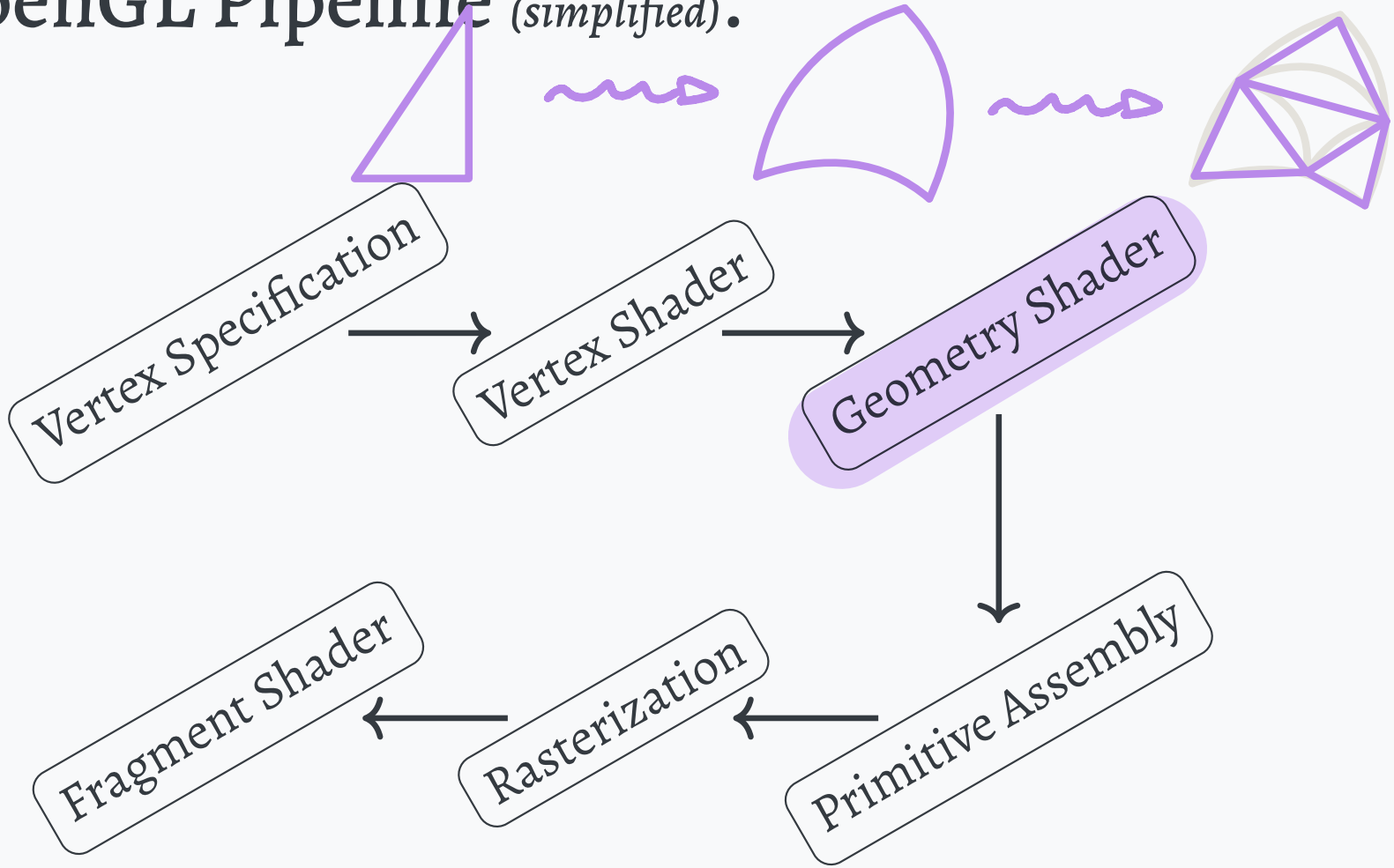
The OpenGL Pipeline *(simplified)*.



The OpenGL Pipeline *(simplified)*.



The OpenGL Pipeline *(simplified)*.



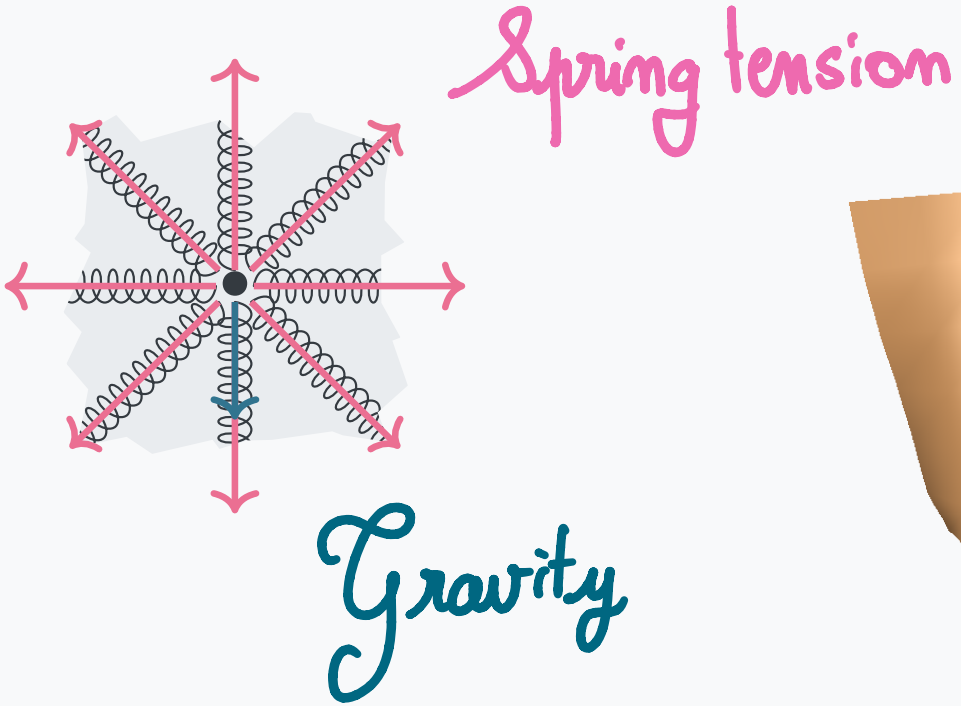
A few examples.

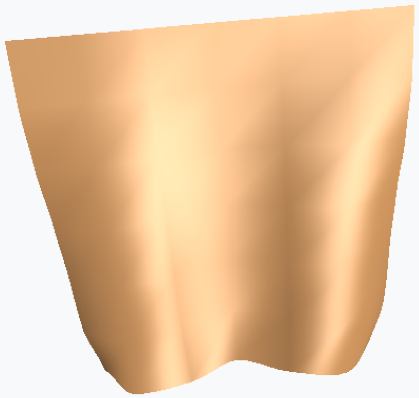
demo

```
python3 opengl_object.py data/suzanne-500.ply
```

A cloth simulation.

demo



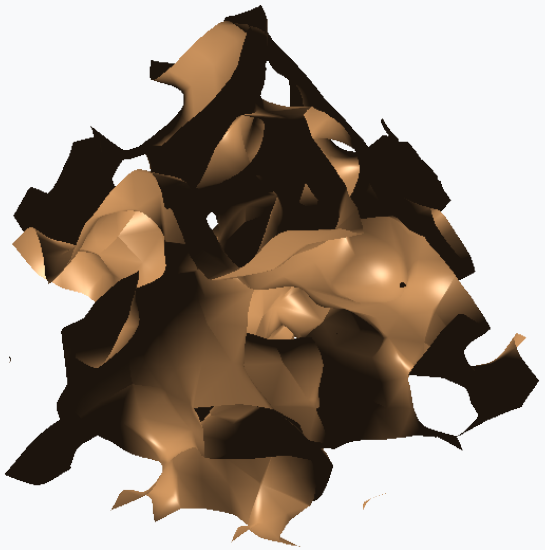


Updating the mesh: $O(n^2)$ where n is the width of the mesh.

```
python3 cloth.py        python3 opengl_cloth.py
```

Marching cubes.

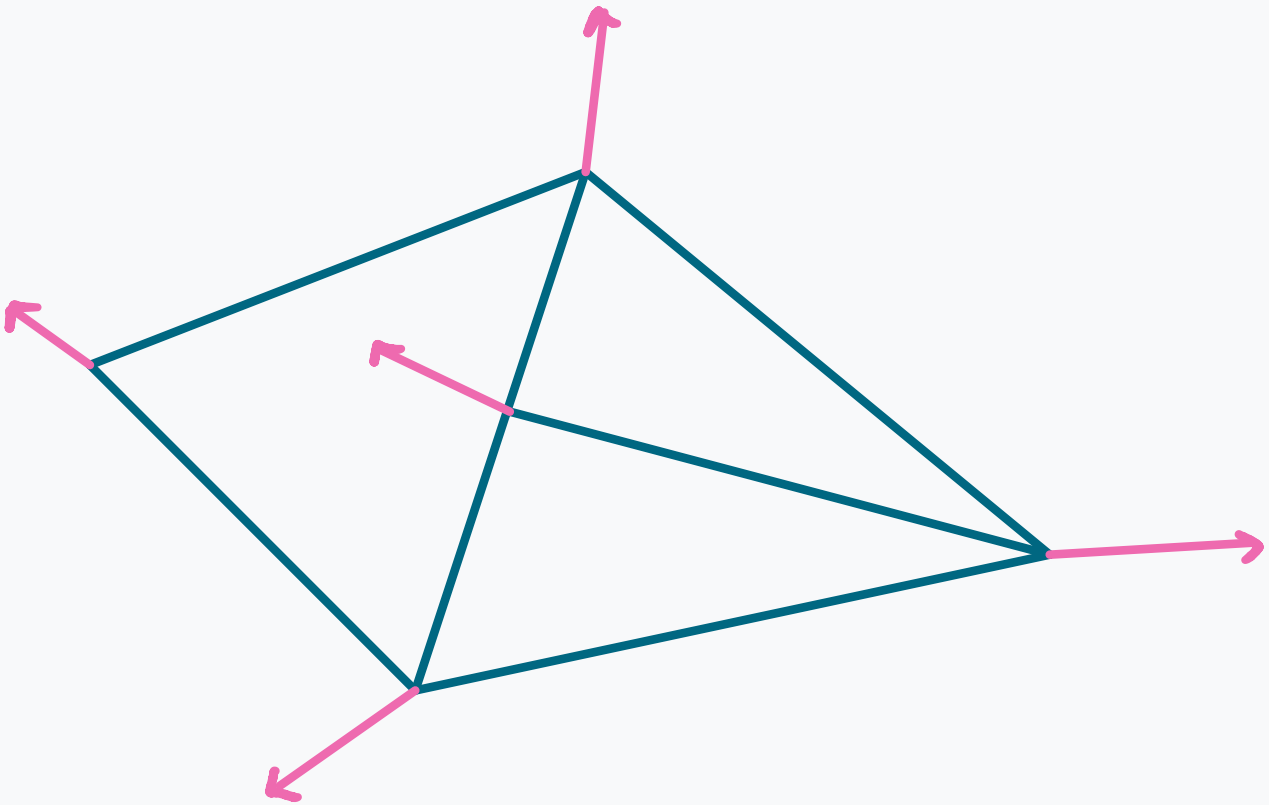
demo



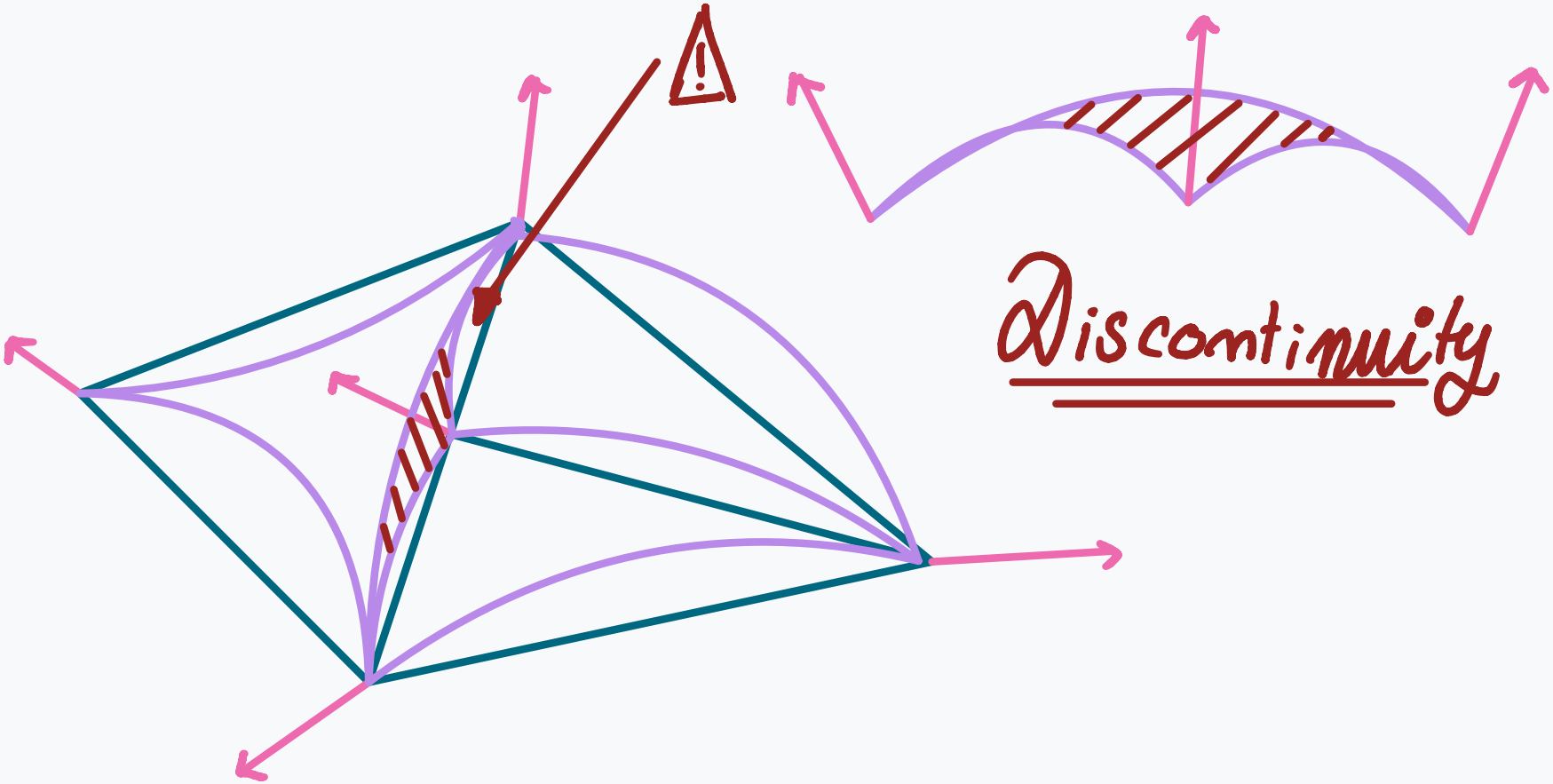
```
python3 cubes.py
```

```
python3 opengl_cubes.py
```

Limits: continuity.



Limits: continuity.



Any questions?